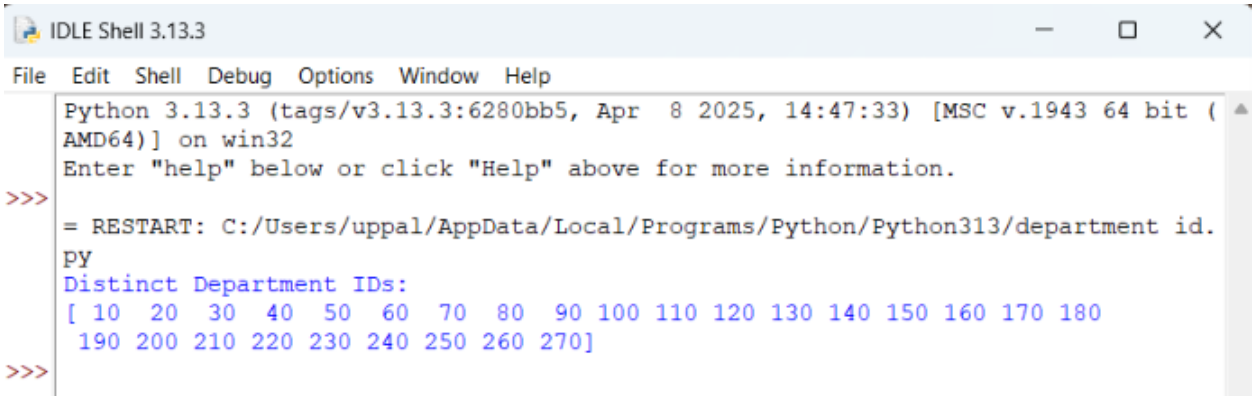
Lab Experiments

1.Python Pandas Program: Select Distinct Department IDs

Code:

```
import pandas as pd
df = pd.DataFrame({
   "DEPARTMENT_ID":
[10,20,30,40,50,60,70,80,90,100,110,120,130,140,150,160,170,180,190,200,210,220,230,240,25
0,260,270],
   "DEPARTMENT_NAME": [
      "Administration","Marketing","Purchasing","Human Resources","Shipping",
      "IT","Public Relations","Sales","Executive","Finance","Accounting",
      "Treasury","Corporate Tax","Control And Credit","Shareholder Services",
      "Benefits","Manufacturing","Construction","Contracting","Operations",
      "IT Support","NOC","IT Helpdesk","Government Sales","Retail Sales",
      "Recruiting","Payroll"
   ],
   "MANAGER_ID":
[200,201,114,203,121,103,204,145,100,108,205,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0],
   "LOCATION_ID":
[1700,1800,1700,2400,1500,1400,2700,2500,1700,1700,1700,1700,1700,1700,1700,1700,1700,
1700,1700,1700,1700,1700,1700,1700,1700,1700,1700]
})
print("Distinct Department IDs:")
print(df["DEPARTMENT_ID"].unique())
```

Output:



2.Display the IDs of Employees Who Have Done Two or More Jobs in the Past Using Pandas

Code:

```
import pandas as pd
df = pd.DataFrame({
    "EMPLOYEE_ID": [102,101,101,201,114,122,200,176,176,200],
    "START_DATE": ["2001-01-13","1997-09-21","2001-10-28","2004-02-17","2006-03-24",
                   "2007-01-01","1995-09-17","2006-03-24","2007-01-01","2002-07-01"],
    "END_DATE": ["2006-07-24","2001-10-27","2005-03-15","2007-12-19","2007-12-31",
                 "2007-12-31","2001-06-17","2006-12-31","2007-12-31","2006-12-31"],
    "JOB_ID": ["IT_PROG","AC_ACCOUNT","AC_MGR","MK_REP","ST_CLERK",
               "ST_CLERK","AD_ASST","SA_REP","SA_MAN","AC_ACCOUNT"],
    "DEPARTMENT_ID": [60,110,110,20,50,50,90,80,80,90]
})
result = df.groupby("EMPLOYEE_ID").filter(lambda x: len(x) >= 2)
print("Employees who have done two or more jobs:")
print(result["EMPLOYEE_ID"].unique())
```

Output:

```
>>>
== RESTART: C:/Users/uppal/AppData/Local/Programs/Python/Python313/group by.py =
Employees who have done two or more jobs:
[101 200 176]
>>>
```

3.Display the Details of Jobs in Descending Order of Job Title Using Pandas

Code:

```
import pandas as pd
df = pd.DataFrame({
   "JOB_ID":
["AD_PRES","AD_VP","AD_ASST","FI_MGR","FI_ACCOUNT","AC_MGR","AC_ACCOU
NT",
"SA_MAN","SA_REP","PU_MAN","PU_CLERK","ST_MAN","ST_CLERK","SH_CLERK",
          "IT_PROG","MK_MAN","MK_REP","HR_REP","PR_REP"],
   "JOB_TITLE": ["President","Administration Vice President","Administration Assistant",
             "Finance Manager","Accountant","Accounting Manager","Public Accountant",
             "Sales Manager","Sales Representative","Purchasing Manager",
             "Purchasing Clerk","Stock Manager","Stock Clerk","Shipping Clerk",
             "Programmer","Marketing Manager","Marketing Representative",
             "Human Resources Representative","Public Relations Representative"],
   "MIN_SALARY":
[20080,15000,3000,8200,4200,8200,4200,10000,6000,8000,2500,5500,2008,2500,4000,9000,40
00,4000,4500],
   "MAX_SALARY":
[40000,30000,6000,16000,9000,16000,9000,20080,12008,15000,5500,8500,5000,5500,10000,1
5000,9000,9000,10500]
})
result = df.sort_values(by="JOB_TITLE", ascending=False)
print(result)
```

Output:

```
>>>
= RESTART: C:/Users/uppal/AppData/Local/Programs/Python/Python313/job title.py =
        JOB_ID                          JOB_TITLE  MIN_SALARY  MAX_SALARY
11      ST_MAN                      Stock Manager        5500        8500
12    ST_CLERK                        Stock Clerk        2008        5000
13    SH_CLERK                    Shipping Clerk        2500        5500
8       SA_REP              Sales Representative        6000       12008
7       SA_MAN                     Sales Manager       10000       20080
9       PU_MAN                Purchasing Manager        8000       15000
10    PU_CLERK                  Purchasing Clerk        2500        5500
18      PR_REP  Public Relations Representative        4500       10500
6   AC_ACCOUNT                 Public Accountant        4200        9000
14     IT_PROG                         Programmer        4000       10000
0      AD_PRES                          President       20080       40000
16      MK_REP         Marketing Representative        4000        9000
15      MK_MAN                Marketing Manager        9000       15000
17      HR_REP   Human Resources Representative        4000        9000
3       FI_MGR                  Finance Manager        8200       16000
1        AD_VP    Administration Vice President       15000       30000
2      AD_ASST         Administration Assistant        3000        6000
5       AC_MGR               Accounting Manager        8200       16000
4   FI_ACCOUNT                        Accountant        4200        9000
>>>
```

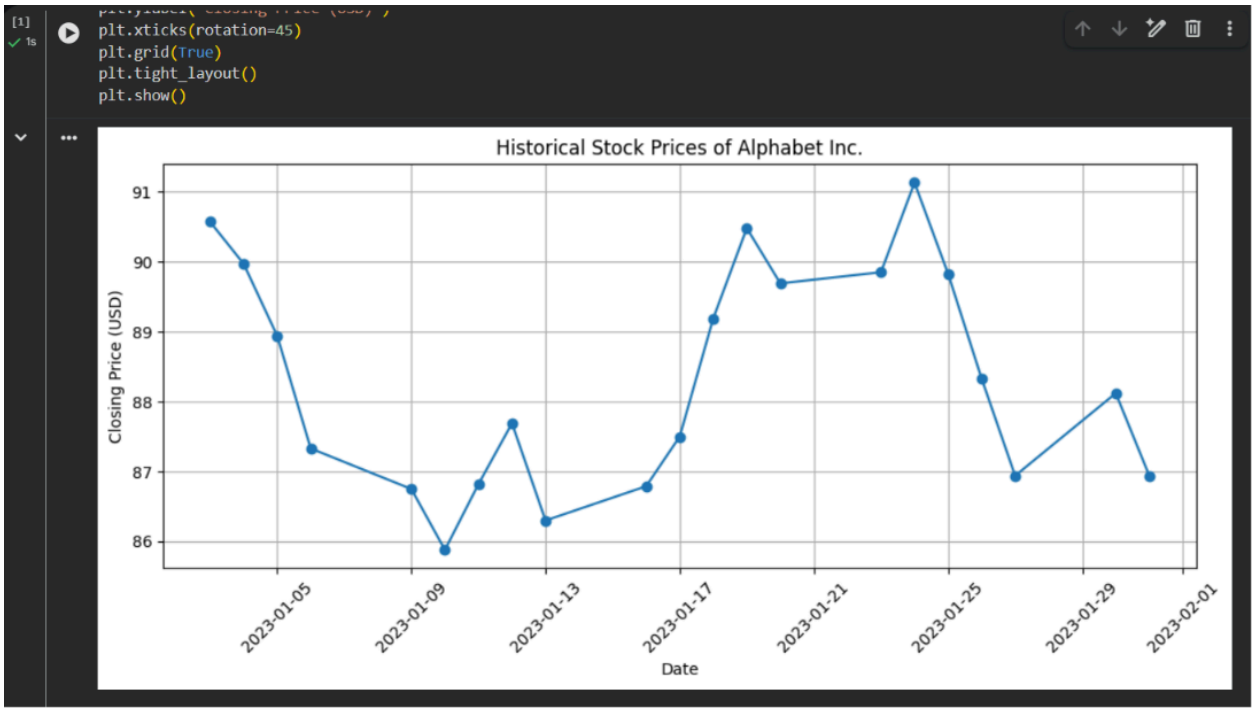
4.Create a Line Plot of Alphabet Inc. Stock Prices Between Two Dates

Python Code:

```
import pandas as pd
import matplotlib.pyplot as plt
df = pd.read_csv("Alphabet_Inc_Historical_Stock_Prices.csv")
df["Date"] = pd.to_datetime(df["Date"])
start_date = "2023-01-03"
end_date = "2023-01-31"
filtered_df = df[(df["Date"] >= start_date) & (df["Date"] <= end_date)]
plt.figure(figsize=(10,5))
plt.plot(filtered_df["Date"], filtered_df["Close"], marker="o")
plt.title("Historical Stock Prices of Alphabet Inc.")
plt.xlabel("Date")
plt.ylabel("Closing Price (USD)")
plt.xticks(rotation=45)
plt.grid(True)
plt.tight_layout()
```

plt.show()

Output:



5. Write a Pandas program to create a bar plot of the trading volume of Alphabet Inc. stock between two specific dates.

Python Code:

```
import pandas as pd
import matplotlib.pyplot as plt
df = pd.read_csv("Alphabet_Trading_Volume.csv")
df["Date"] = pd.to_datetime(df["Date"])
start_date = "2023-02-01"
end_date = "2023-02-28"
filtered_df = df[(df["Date"] >= start_date) & (df["Date"] <= end_date)]
plt.figure(figsize=(10,5))
plt.bar(filtered_df["Date"].dt.strftime("%Y-%m-%d"), filtered_df["Volume"])
```

plt.title("Trading Volume of Alphabet Inc. Stock")

plt.xlabel("Date")

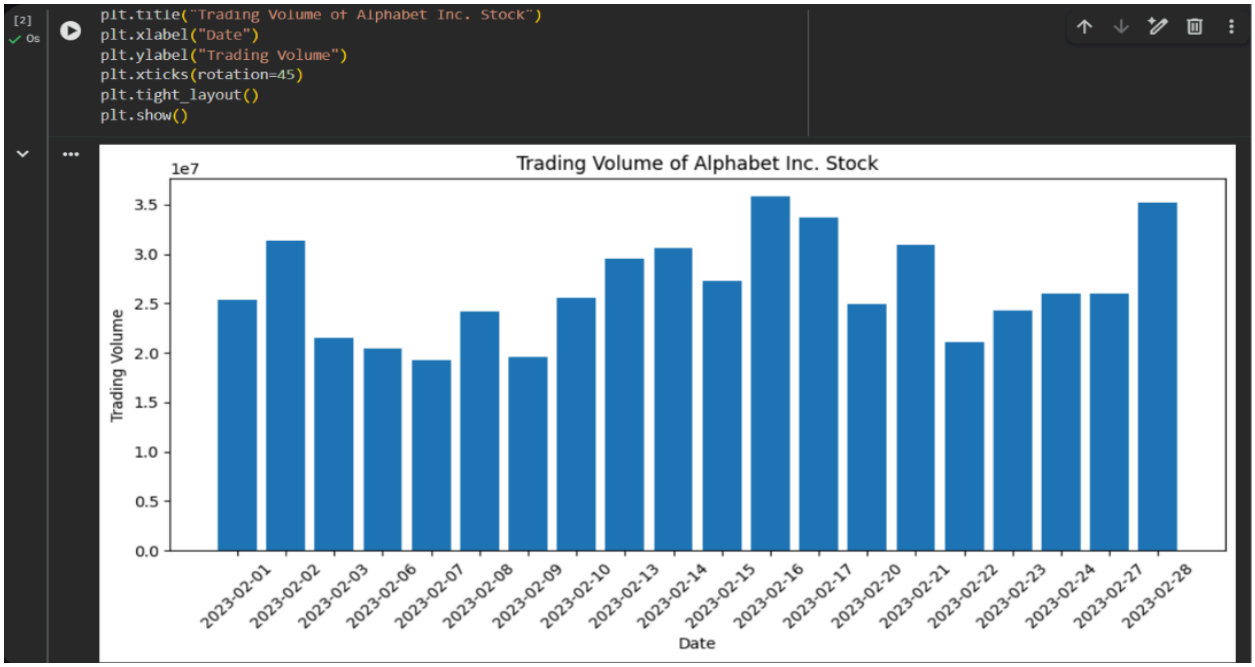
plt.ylabel("Trading Volume")

plt.xticks(rotation=45)

plt.tight_layout()

plt.show()

Output:



6 . A scatter plot of the trading volume of Alphabet Inc. stock between two specific dates.

Python Code:

import pandas as pd

import matplotlib.pyplot as plt

df = pd.read_csv("Alphabet_Inc_Trading_Volume_Dataset.csv")

df["Date"] = pd.to_datetime(df["Date"])

start_date = "2023-02-01"

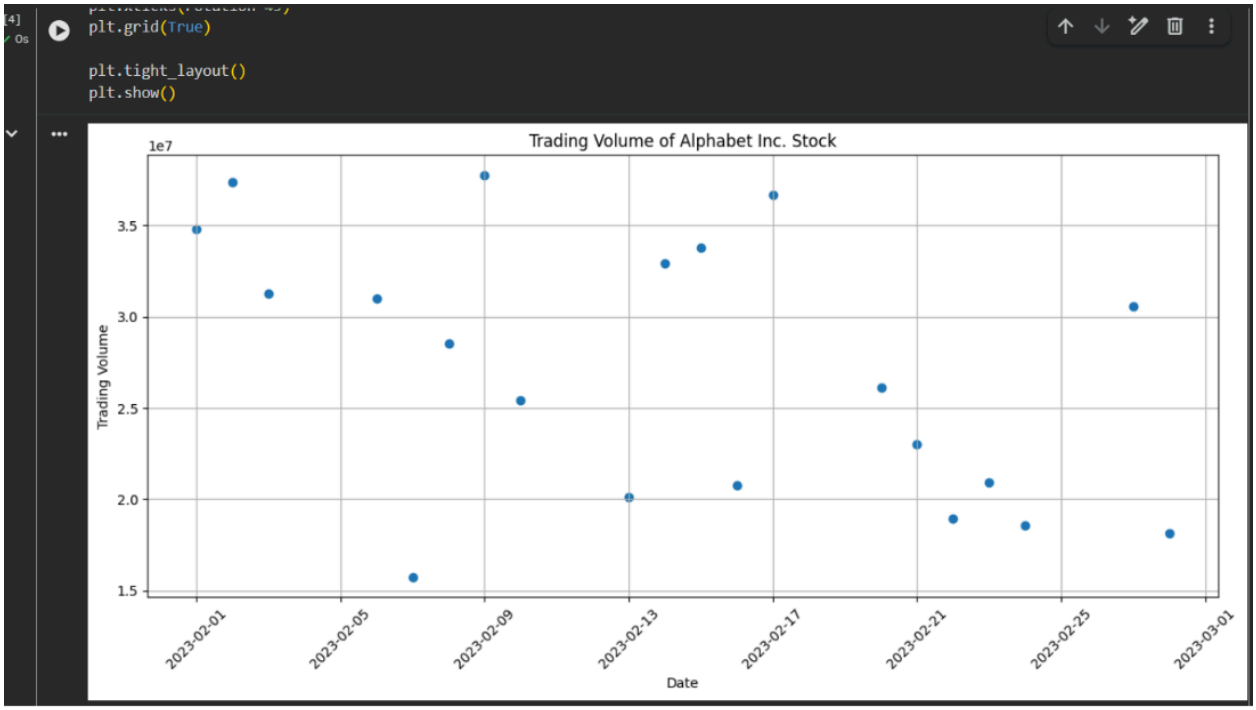
end_date = "2023-02-28"

filtered_df = df[(df["Date"] >= start_date) & (df["Date"] <= end_date)]

plt.figure(figsize=(12,6))

```
plt.scatter(filtered_df["Date"], filtered_df["Volume"])
plt.title("Trading Volume of Alphabet Inc. Stock")
plt.xlabel("Date")
plt.ylabel("Trading Volume")
plt.xticks(rotation=45)
plt.grid(True)
plt.tight_layout()
plt.show()
```

Output:



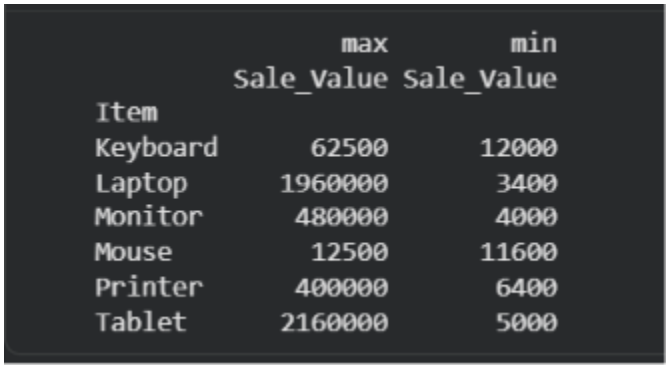
7. Create a Pivot table and find the maximum and minimum sale value of the items.(refer sales_data table)

Python Code:

```
import pandas as pd
df = pd.read_csv("sales_data.csv")
pivot = pd.pivot_table(
    df,
    values="Sale_Value",
```

```
    index="Item",
    aggfunc=["max", "min"]
)
print(pivot)
```

Output:



Item	max Sale_Value	min Sale_Value
Keyboard	62500	12000
Laptop	1960000	3400
Monitor	480000	4000
Mouse	12500	11600
Printer	400000	6400
Tablet	2160000	5000

8. Create a Pivot table and find the item wise unit sold. .(refer sales_data table)

Python Code:

```
import pandas as pd
pivot = pd.pivot_table(
    df,
    values="Units_Sold",
    index="Item",
    aggfunc="sum"
)
print(pivot)
```

Output:

```
...            Units_Sold
    Item
    Keyboard           88
    Laptop            202
    Monitor            40
    Mouse              63
    Printer            76
    Tablet            419
```

9.Find the total sale amount region-wise, manager-wise, and salesman-wise. (Refer Sales_data table)

Python Code:

```
import pandas as pd
df = pd.read_csv("Sales_data.csv")
pivot = pd.pivot_table(
    df,
    values="Sale_amt",
    index=["Region", "Manager", "SalesMan"],
    aggfunc="sum"
)
print(pivot)
```

Output:

```
...                              Sale_amt
    Region  Manager SalesMan
    Central Douglas John           250.0
            Hermann Luis        150948.0
                    Shelli       25000.0
                    Sigal       121820.0
            Martha  Steven       89850.0
            Timothy David         6075.0
    East    Douglas Karen        40500.0
            Martha  Alexander   231076.0
                    Diana        14500.0
    West    Douglas Michael      38336.0
            Timothy Stephen      67088.0
```

10.To highlight the negative numbers in red and positive numbers in black.

Python Code:

```
import pandas as pd
import numpy as np
np.random.seed(10)
df = pd.DataFrame(
    np.random.randn(10, 5),
    columns=['A', 'B', 'C', 'D', 'E']
)
def highlight_numbers(value):
    if value < 0:
        return 'color: red'
    else:
        return 'color: black'
styled_df = df.style.map(highlight_numbers)
styled_df
```

Output:

	A	B	C	D	E
0	1.331587	0.715279	-1.545400	-0.008384	0.621336
1	-0.720086	0.265512	0.108549	0.004291	-0.174600
2	0.433026	1.203037	-0.965066	1.028274	0.228630
3	0.445138	-1.136602	0.135137	1.484537	-1.079805
4	-1.977728	-1.743372	0.266070	2.384967	1.123691
5	1.672622	0.099149	1.397996	-0.271248	0.613204
6	-0.267317	-0.549309	0.132708	-0.476142	1.308473
7	0.195013	0.400210	-0.337632	1.256472	-0.731970
8	0.660232	-0.350872	-0.939433	-0.489337	-0.804591
9	-0.212698	-0.339140	0.312170	0.565153	-0.147420

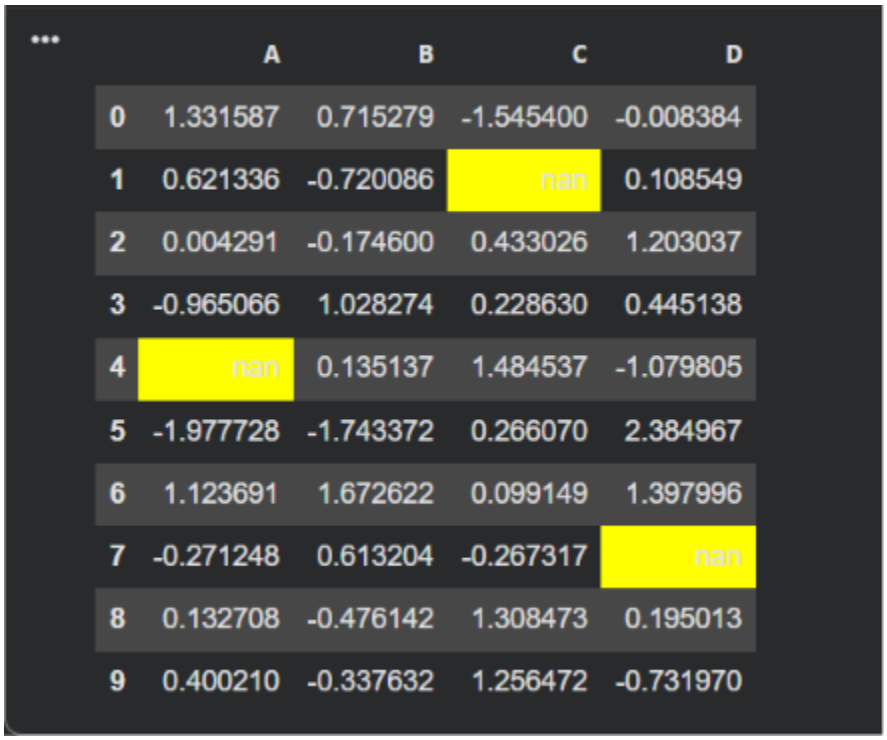
11.Highlight NaN Values in a DataFrame

Python Code:

```
import pandas as pd
import numpy as np
np.random.seed(10)
df = pd.DataFrame(
    np.random.randn(10, 4),
    columns=['A', 'B', 'C', 'D']
)
df.iloc[1, 2] = np.nan
df.iloc[4, 0] = np.nan
df.iloc[7, 3] = np.nan
def highlight_nan(value):
    if pd.isna(value):
        return 'background-color: yellow'
    return ''
```

```
styled_df = df.style.map(highlight_nan)
styled_df
```

Output:



	A	B	C	D
0	1.331587	0.715279	-1.545400	-0.008384
1	0.621336	-0.720086	nan	0.108549
2	0.004291	-0.174600	0.433026	1.203037
3	-0.965066	1.028274	0.228630	0.445138
4	nan	0.135137	1.484537	-1.079805
5	-1.977728	-1.743372	0.266070	2.384967
6	1.123691	1.672622	0.099149	1.397996
7	-0.271248	0.613204	-0.267317	nan
8	0.132708	-0.476142	1.308473	0.195013
9	0.400210	-0.337632	1.256472	-0.731970

12.Set DataFrame Background Black and Font Color Yellow

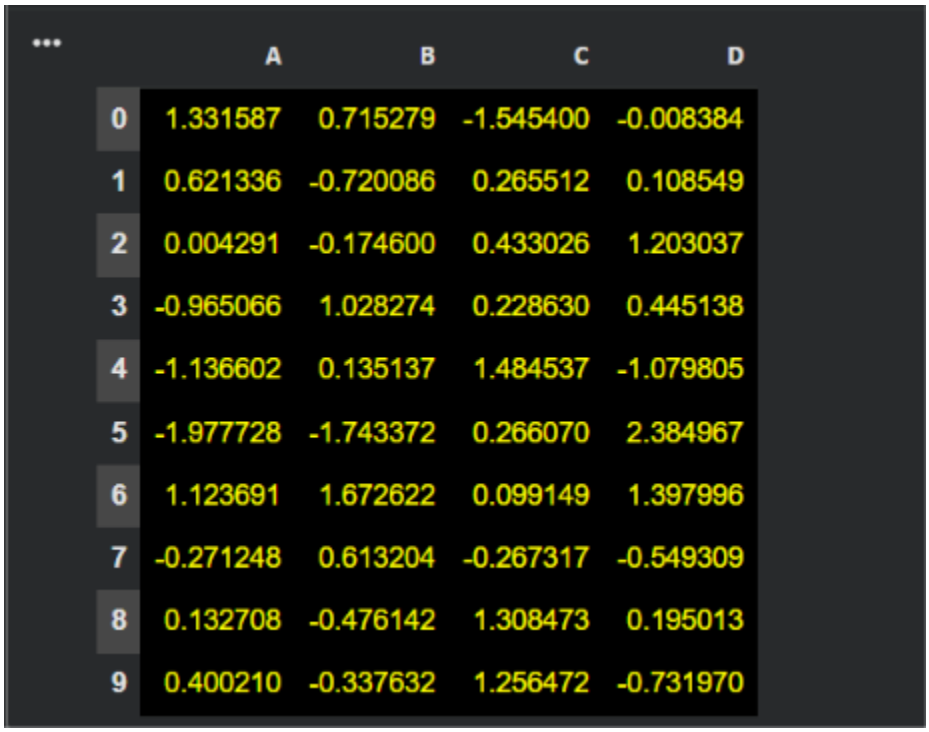
Python Code:

```
import pandas as pd
import numpy as np
np.random.seed(10)
df = pd.DataFrame(
    np.random.randn(10, 4),
    columns=['A', 'B', 'C', 'D']
)
styled_df = df.style.set_properties(**{
    'background-color': 'black',
    'color': 'yellow'
```

})

styled_df

Output:



	A	B	C	D
0	1.331587	0.715279	-1.545400	-0.008384
1	0.621336	-0.720086	0.265512	0.108549
2	0.004291	-0.174600	0.433026	1.203037
3	-0.965066	1.028274	0.228630	0.445138
4	-1.136602	0.135137	1.484537	-1.079805
5	-1.977728	-1.743372	0.266070	2.384967
6	1.123691	1.672622	0.099149	1.397996
7	-0.271248	0.613204	-0.267317	-0.549309
8	0.132708	-0.476142	1.308473	0.195013
9	0.400210	-0.337632	1.256472	-0.731970

13.Detect Missing Values in a DataFrame

Python Code:

```
import pandas as pd
import numpy as np
df = pd.DataFrame({
    'ord_no': [70001, np.nan, 70002, 70004, np.nan],
    'purch_amt': [150.50, 270.65, 65.26, 110.50, 948.50],
    'ord_date': ['2012-10-05', '2012-09-10', np.nan, '2012-08-17', '2012-09-10'],
    'customer_id': [3002, 3001, 3001, 3003, 3002],
    'salesman_id': [5002, 5003, 5001, np.nan, 5002]
})
print(df.isnull())
```

Output:

	ord_no	purch_amt	ord_date	customer_id	salesman_id
0	False	False	False	False	False
1	True	False	False	False	False
2	False	False	True	False	False
3	False	False	False	False	True
4	True	False	False	False	False

14.Find and Replace Missing Values in a DataFrame

Python Code:

```
import pandas as pd
df = pd.DataFrame({
    'ord_no': [70001, np.nan, 70002, 70004, np.nan],
    'purch_amt': [150.50, 270.65, np.nan, 110.50, 948.50],
    'ord_date': ['2012-10-05', '2012-09-10', np.nan, '2012-08-17', '2012-09-10'],
    'customer_id': [3002, 3001, 3001, np.nan, 3002],
    'salesman_id': [5002, 5003, 5001, np.nan, 5002]
})
df = df.fillna("Not Available")
print(df)
```

Output:

	ord_no	purch_amt	ord_date	customer_id	salesman_id
0	70001.0	150.5	2012-10-05	3002.0	5002.0
1	Not Available	270.65	2012-09-10	3001.0	5003.0
2	70002.0	Not Available	Not Available	3001.0	5001.0
3	70004.0	110.5	2012-08-17	Not Available	Not Available
4	Not Available	948.5	2012-09-10	3002.0	5002.0

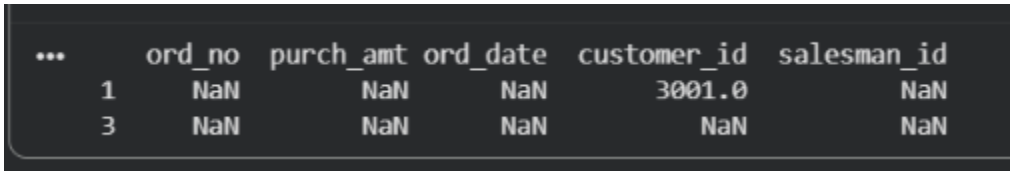
15.Keep Rows with at Least 2 NaN Values

Python Code:

```
import pandas as pd
import numpy as np
df = pd.DataFrame({
```

```
    'ord_no': [70001, np.nan, 70002, np.nan, 70005],
    'purch_amt': [150.50, np.nan, 65.26, np.nan, 2400.60],
    'ord_date': ['2012-10-05', np.nan, '2012-09-10', np.nan, '2012-07-27'],
    'customer_id': [3002, 3001, np.nan, np.nan, 3001],
    'salesman_id': [5002, np.nan, 5001, np.nan, 5001]
})
result = df[df.isnull().sum(axis=1) >= 2]
print(result)
```

Output:



```
...     ord_no  purch_amt ord_date  customer_id  salesman_id
1      NaN         NaN      NaN       3001.0          NaN
3      NaN         NaN      NaN          NaN          NaN
```

16.Split DataFrame into Groups by School Code

Python Code:

```
import pandas as pd
df = pd.read_csv("school_data.csv")
groups = df.groupby("school_code")
for name, group in groups:
    print("\nSchool Code:", name)
    print(group)
```

Output:

```
School Code: 1001
  school  school_code class           name date_of_birth  age  height  \
0     S1         1001     V  Alberto Franco    15/05/2002   12     173

   weight  address
0      35  street1

School Code: 1002
  school  school_code class         name date_of_birth  age  height  weight  \
1     S2         1002     V  Gino Mcneill    17/05/1998   12     192      32

   address
1  street2

School Code: 1003
  school  school_code class         name date_of_birth  age  height  weight  \
2     S3         1003    VI  Ryan Parkes    15/09/1999   13     186      33

   address
2  street3

School Code: 1004
  school  school_code class         name date_of_birth  age  height  weight  \
3     S4         1004    VI  Eesha Hinton    25/09/1998   13     167      30

   address
3  street1

School Code: 1005
  school  school_code class         name date_of_birth  age  height  weight  \
4     S5         1005   VII  Gina Howell    11/03/1997   14     151      31

   address
4  street2

School Code: 1006
  school  school_code class         name date_of_birth  age  height  weight  \
5     S6         1006   VII  David Parkes    15/09/1997   12     159      32

   address
5  street4
```

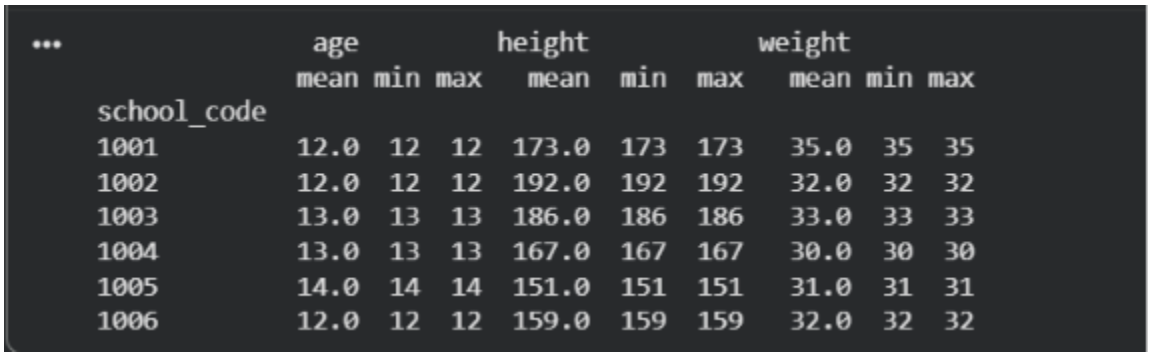
17.Get Mean, Min and Max Values by School Code

Python Code:

```
import pandas as pd
df = pd.read_csv("school_data.csv")
result = df.groupby("school_code")[["age", "height", "weight"]].agg(["mean", "min", "max"])
```

print(result)

Output:



	age			height			weight		
	mean	min	max	mean	min	max	mean	min	max
school_code									
1001	12.0	12	12	173.0	173	173	35.0	35	35
1002	12.0	12	12	192.0	192	192	32.0	32	32
1003	13.0	13	13	186.0	186	186	33.0	33	33
1004	13.0	13	13	167.0	167	167	30.0	30	30
1005	14.0	14	14	151.0	151	151	31.0	31	31
1006	12.0	12	12	159.0	159	159	32.0	32	32

18. Split DataFrame into groups based on school code and class

Python Code:

```
import pandas as pd
data = {
   'school': ['s001','s002','s003','s001','s002','s004'],
   'class': ['V','V','VI','VI','V','VI'],
   'name': ['Alberto Franco','Gino McNeill','Ryan Parkes',
         'Esha Hinton','Gino McNeill','David Parkes'],
   'age': [12,13,11,13,14,12],
   'height': [159,192,168,187,151,159],
   'weight': [35,32,33,30,31,32]
}
df = pd.DataFrame(data)
groups = df.groupby(['school', 'class'])
for name, group in groups:
   print(name)
   print(group)
```

Output:

```
('s001', 'V')
  school class           name  age  height  weight
0   s001     V  Alberto Franco   12     159      35
('s001', 'VI')
  school class         name  age  height  weight
3   s001    VI  Esha Hinton   13     187      30
('s002', 'V')
  school class          name  age  height  weight
1   s002     V  Gino McNeill   13     192      32
4   s002     V  Gino McNeill   14     151      31
('s003', 'VI')
  school class         name  age  height  weight
2   s003    VI  Ryan Parkes   11     168      33
('s004', 'VI')
  school class          name  age  height  weight
5   s004    VI  David Parkes   12     159      32
```

19. Display dimensions/shape and column names of World Alcohol Consumption dataset

Python Code:

```
import pandas as pd
data = {
   'Year': [1986, 1986, 1985, 1986, 1987],
   'WHO region': ['Western Pacific', 'Americas', 'Africa', 'Americas', 'Americas'],
   'Country': ['Viet Nam', 'Uruguay', 'Congo', 'Colombia', 'Chile'],
   'Beverage Types': ['Wine', 'Beer', 'Wine', 'Beer', 'Beer'],
   'Display Value': [0.00, 0.50, 1.62, 4.27, 1.50]
}
df = pd.DataFrame(data)
print("Shape:", df.shape)
print("Column names:")
print(df.columns)
```

Output:

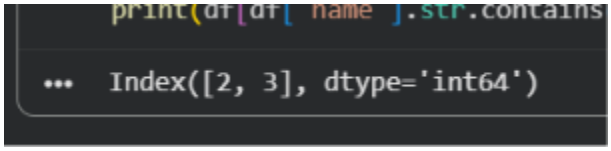
```
Shape: (5, 5)
Column names:
Index(['Year', 'WHO region', 'Country', 'Beverage Types', 'Display Value'], dtype='object')
```

20. Find the index of a given substring in a DataFrame column

Python Code:

```
import pandas as pd
df = pd.DataFrame({
   'name': ['Alberto Franco', 'Gino McNeill',
            'Ryan Parkes', 'David Parkes']
})
print(df[df['name'].str.contains('Parkes')].index)
```

Output:

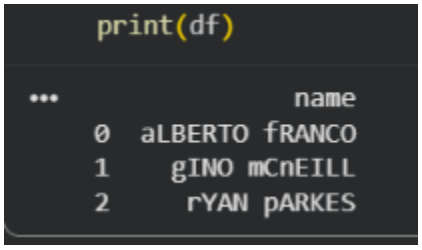


21. Swap the cases of a specified character column

Python Code:

```
import pandas as pd
df = pd.DataFrame({
   'name': ['Alberto Franco', 'Gino McNeill', 'Ryan Parkes']
})
df['name'] = df['name'].str.swapcase()
print(df)
```

Output:

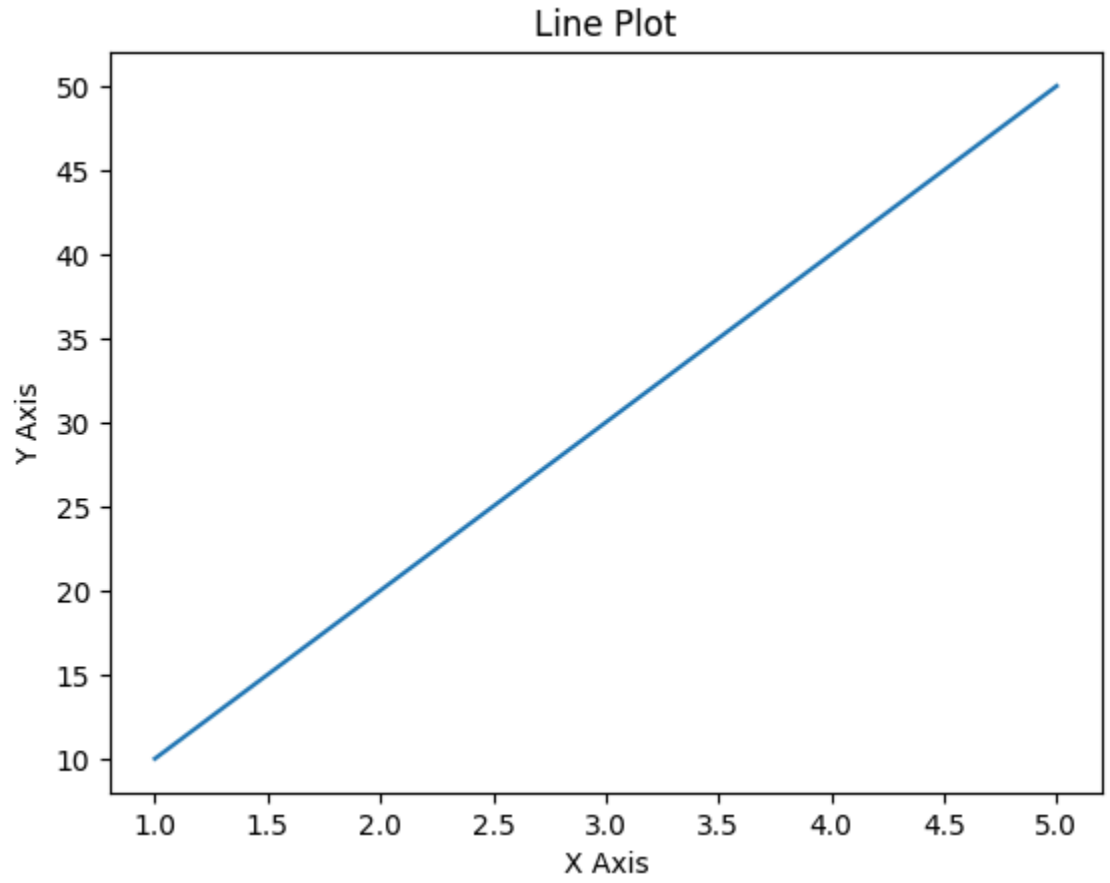


22. Draw a line with labels on X-axis, Y-axis and title

Python Code:

```
import matplotlib.pyplot as plt
x = [1, 2, 3, 4, 5]
y = [10, 20, 30, 40, 50]
plt.plot(x, y)
plt.xlabel("X Axis")
plt.ylabel("Y Axis")
plt.title("Line Plot")
plt.show()
```

Output:

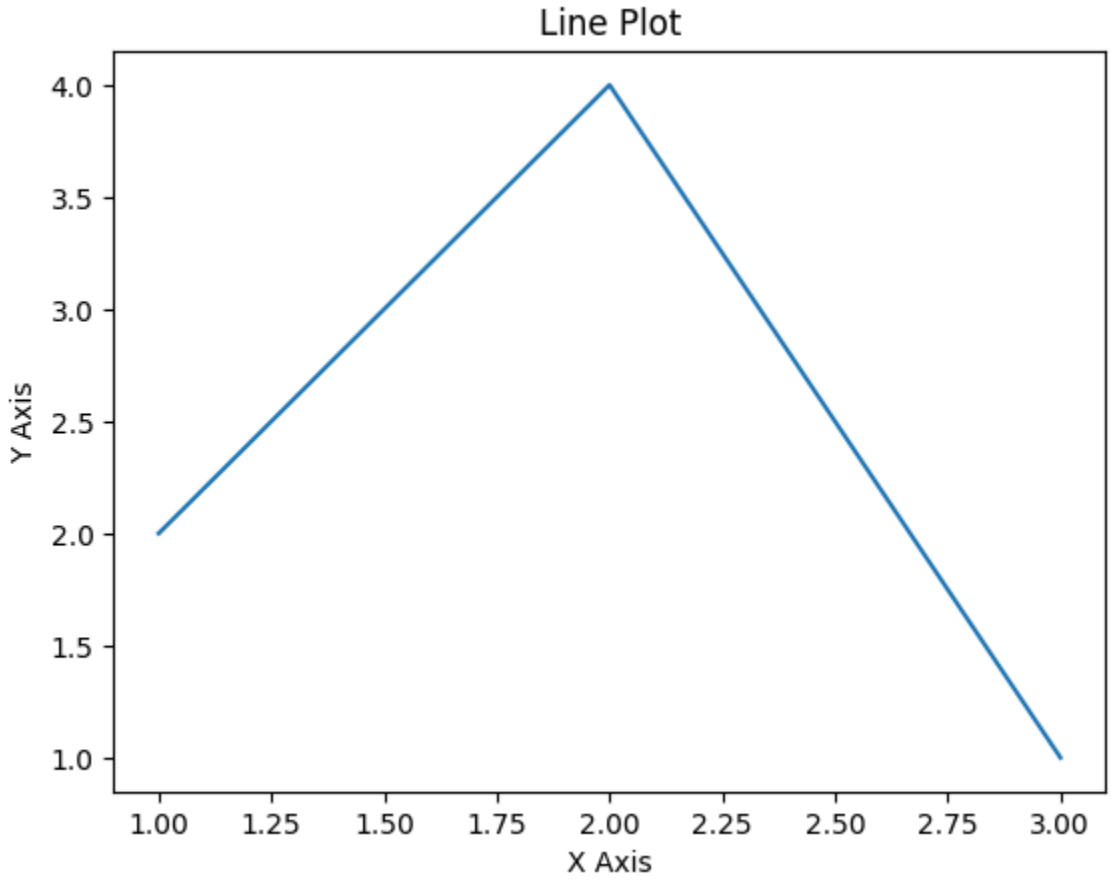


23. Draw a line using X and Y values from a text file

Python Code:

```
import matplotlib.pyplot as plt
x = [1, 2, 3]
y = [2, 4, 1]
plt.plot(x, y)
plt.xlabel("X Axis")
plt.ylabel("Y Axis")
plt.title("Line Plot")
plt.show()
```

Output:



24.To draw line charts of the financial data of Alphabet

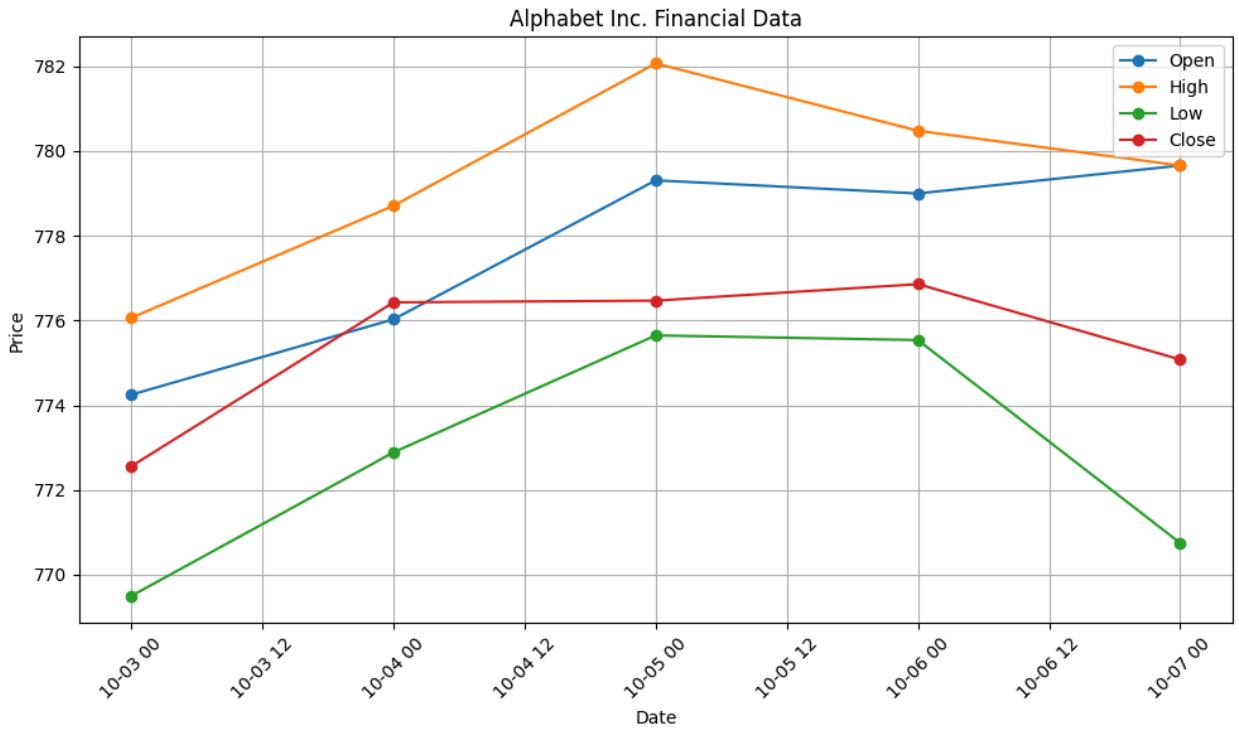
Python Code:

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
df = pd.read_csv("fdata.csv")
df["Date"] = pd.to_datetime(df["Date"], format="%m-%d-%y")
print(df)
plt.figure(figsize=(10, 6))
plt.plot(df["Date"], df["Open"], marker="o", label="Open")
plt.plot(df["Date"], df["High"], marker="o", label="High")
plt.plot(df["Date"], df["Low"], marker="o", label="Low")
plt.plot(df["Date"], df["Close"], marker="o", label="Close")
plt.title("Alphabet Inc. Financial Data")
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.grid(True)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

Output:

```
...        Date        Open        High         Low       Close
0 2016-10-03  774.250000  776.065002  769.500000  772.559998
1 2016-10-04  776.030029  778.710022  772.890015  776.429993
2 2016-10-05  779.309998  782.070007  775.650024  776.469971
3 2016-10-06  779.000000  780.479980  775.539978  776.859985
4 2016-10-07  779.659973  779.659973  770.750000  775.080017
```

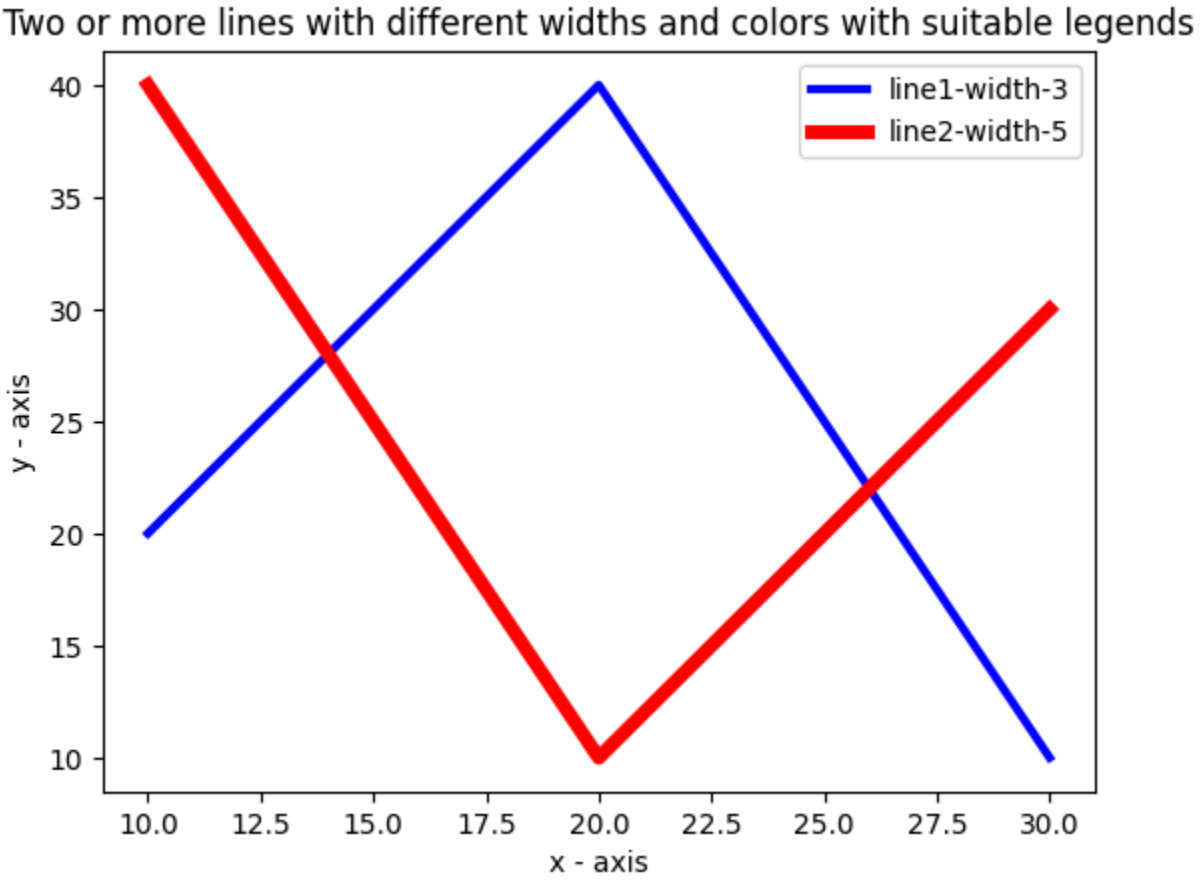


25. Plotting Multiple Lines with Legends, Colors, and Different Line Widths

Python Code:

```
import matplotlib.pyplot as plt
x = [10, 20, 30]
y1 = [20, 40, 10]
y2 = [40, 10, 30]
plt.plot(x, y1, color="blue", linewidth=3,
        label="line1-width-3")
plt.plot(x, y2, color="red", linewidth=5,
        label="line2-width-5")
plt.title("Two or more lines with different widths and colors with suitable legends")
plt.xlabel("x - axis")
plt.ylabel("y - axis")
plt.legend()
plt.show()
```

Output:



26. Create Multiple Plots

Python Code:

```
import matplotlib.pyplot as plt
plt.subplot(2, 1, 1)
plt.plot([1, 2, 3, 4], [10, 20, 30, 40])
plt.title("Line Plot")
plt.subplot(2, 3, 4)
plt.bar([1, 2, 3], [10, 20, 15])
plt.title("Bar Plot")
plt.subplot(2, 3, 5)
plt.scatter([1, 2, 3], [10, 20, 15])
plt.title("Scatter Plot")
plt.subplot(2, 3, 6)
```

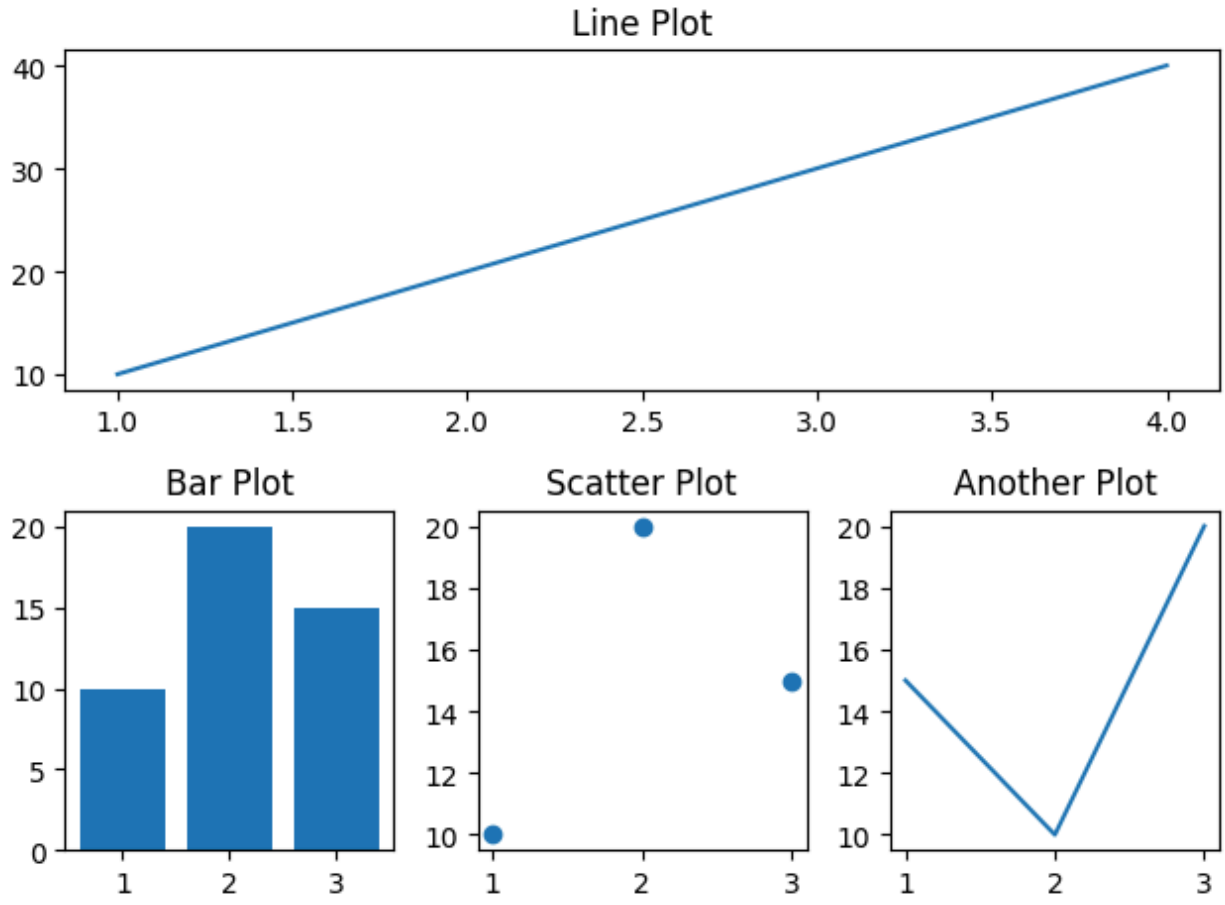
plt.plot([1, 2, 3], [15, 10, 20])

plt.title("Another Plot")

plt.tight_layout()

plt.show()

Output:



27. Display a Vertical Bar Chart of Programming Languages

Python Code:

import matplotlib.pyplot as plt

languages = ["Java", "Python", "PHP", "JavaScript", "C#", "C++"]

popularity = [22, 17, 16, 8, 8, 7]

plt.bar(languages, popularity, color="blue")

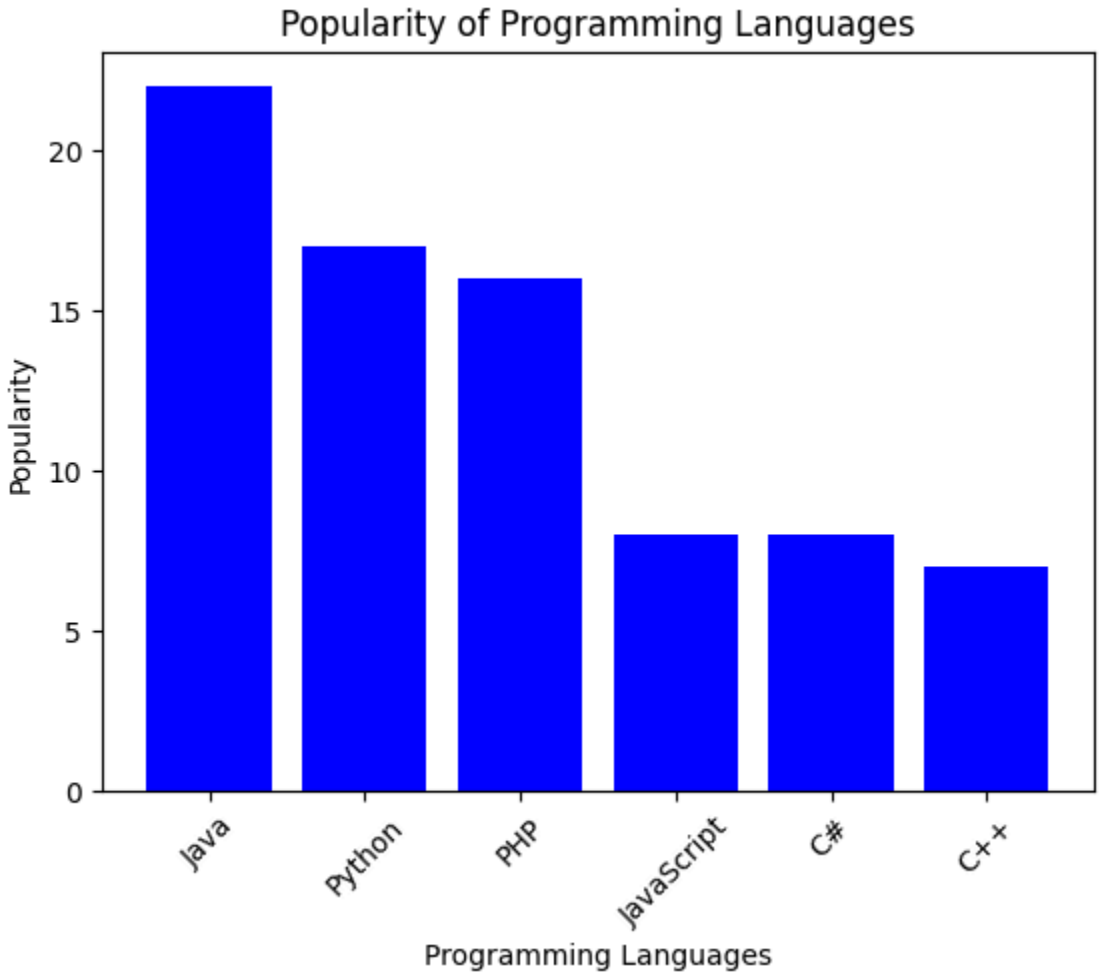
plt.title("Popularity of Programming Languages")

plt.xlabel("Programming Languages")

plt.ylabel("Popularity")

```
plt.xticks(rotation=45)
plt.show()
```

Output:



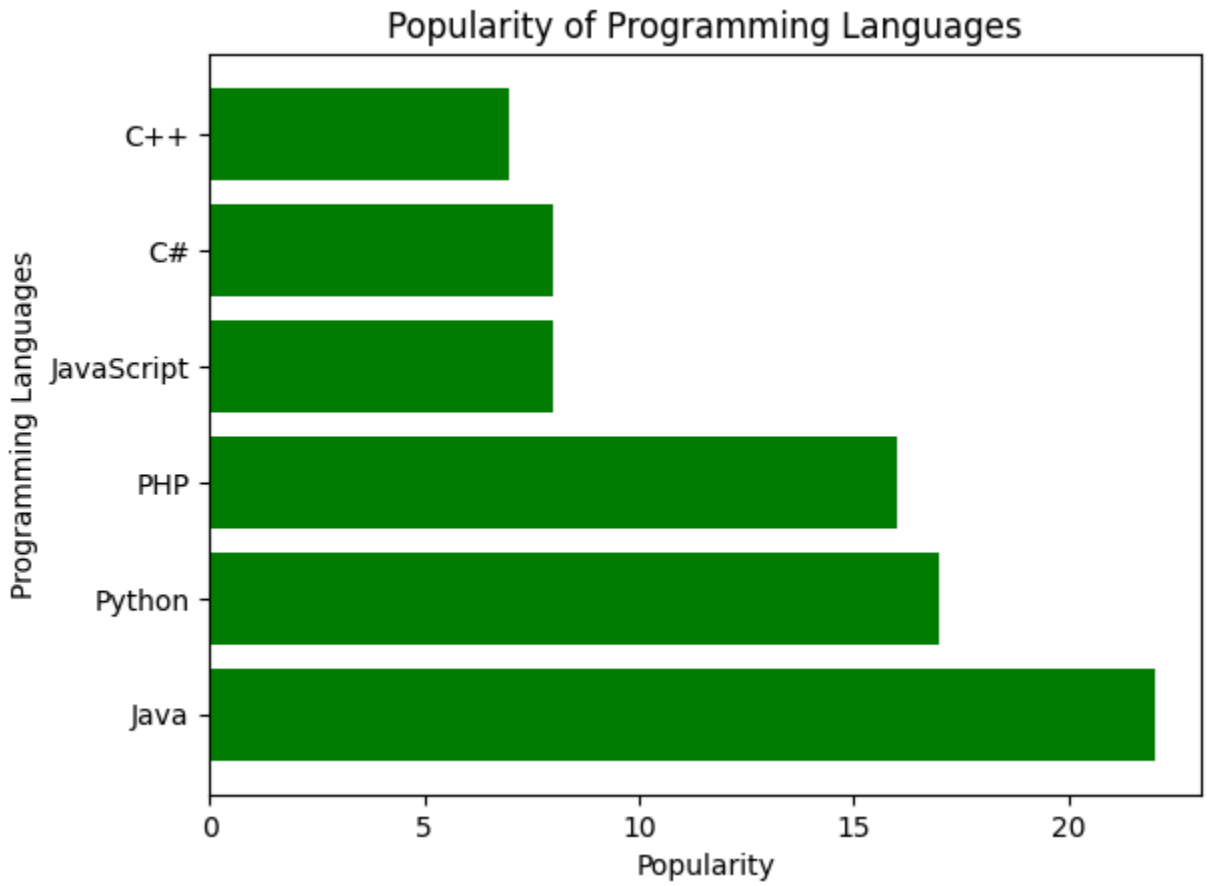
28. Display a Horizontal Bar Chart

Python Code:

```
import matplotlib.pyplot as plt
languages = ["Java", "Python", "PHP", "JavaScript", "C#", "C++"]
popularity = [22, 17, 16, 8, 8, 7]
plt.barh(languages, popularity, color="green")
plt.title("Popularity of Programming Languages")
plt.xlabel("Popularity")
```

```
plt.ylabel("Programming Languages")
plt.show()
```

Output:



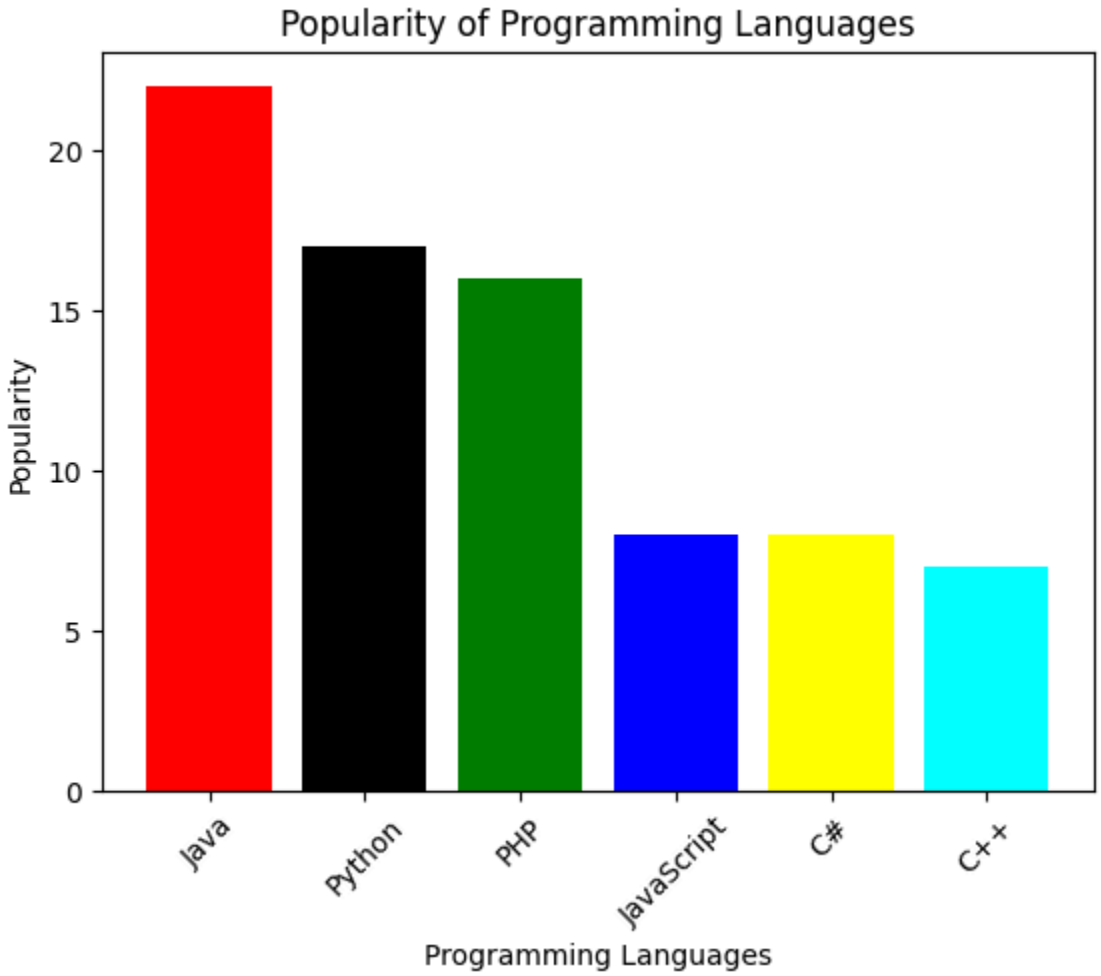
29. Bar Chart with Different Colors

Python Code:

```
import matplotlib.pyplot as plt
languages = ["Java", "Python", "PHP", "JavaScript", "C#", "C++"]
popularity = [22, 17, 16, 8, 8, 7]
colors = ["red", "black", "green", "blue", "yellow", "cyan"]
plt.bar(languages, popularity, color=colors)
plt.title("Popularity of Programming Languages")
plt.xlabel("Programming Languages")
```

```
plt.ylabel("Popularity")
plt.xticks(rotation=45)
plt.show()
```

Output:



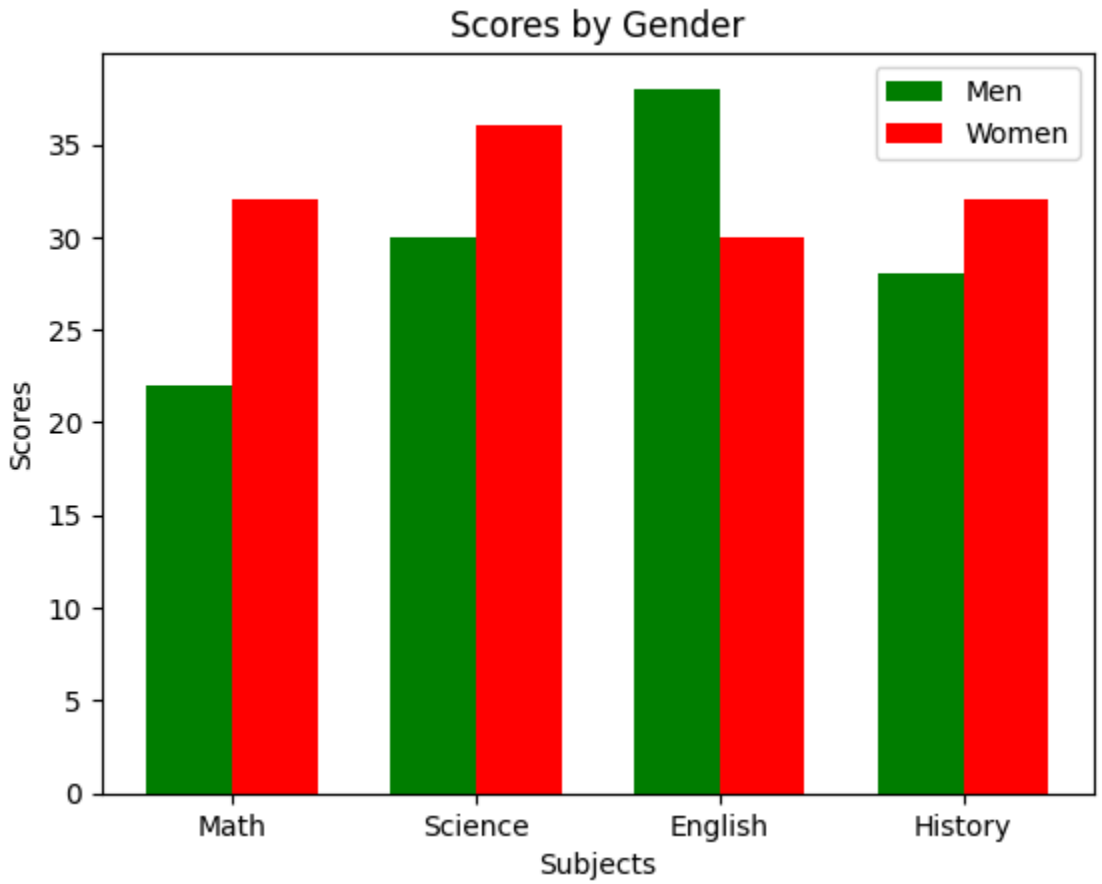
30. Grouped Bar Chart with Error Bars

Python Code:

```
import numpy as np
import matplotlib.pyplot as plt
categories = ["Math", "Science", "English", "History"]
men_means = [22, 30, 38, 28]
women_means = [32, 36, 30, 32]
```

```
x = np.arange(len(categories))
width = 0.35
plt.bar(x - width/2, men_means, width,
        color="green", label="Men")
plt.bar(x + width/2, women_means, width,
        color="red", label="Women")
plt.xlabel("Subjects")
plt.ylabel("Scores")
plt.title("Scores by Gender")
plt.xticks(x, categories)
plt.legend()
plt.show()
```

Output:

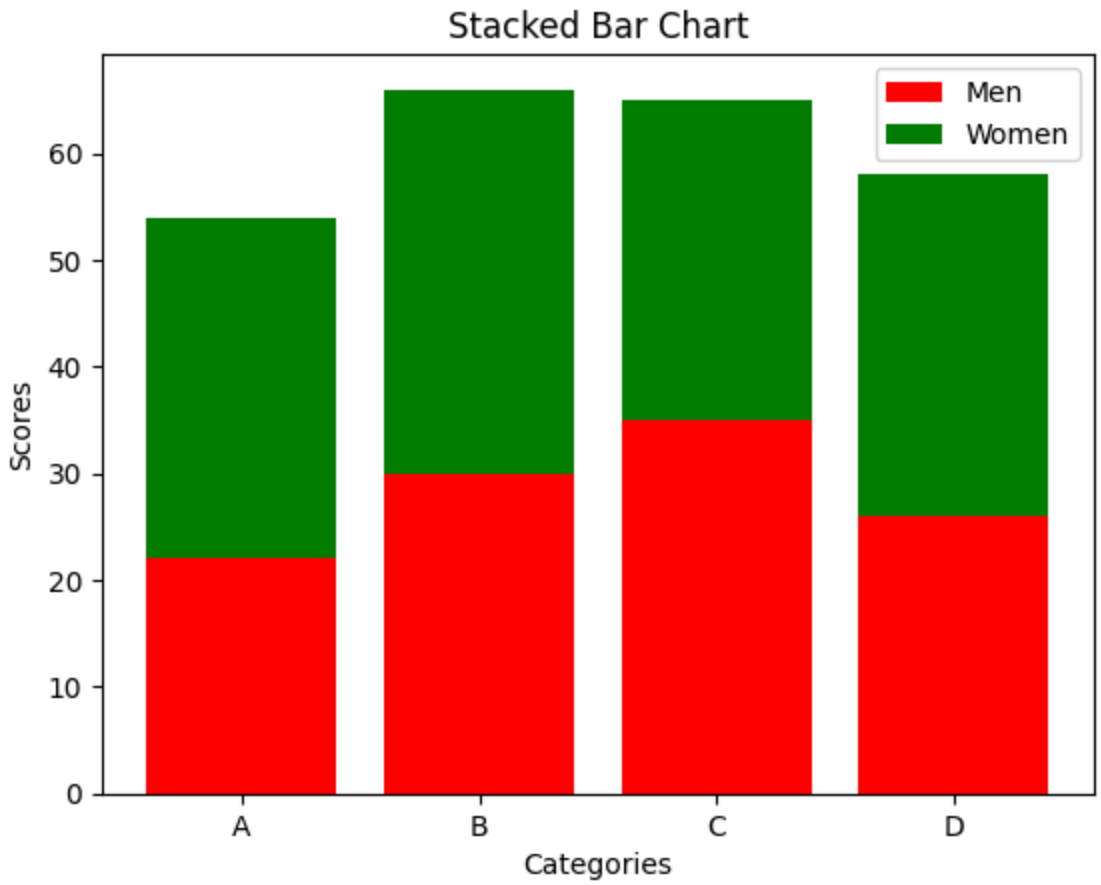


31. Stacked Bar Chart with Error Bars

Python Code:

```
import numpy as np
import matplotlib.pyplot as plt
categories = ["A", "B", "C", "D"]
men = [22, 30, 35, 26]
women = [32, 36, 30, 32]
x = np.arange(len(categories))
plt.bar(x, men, label="Men", color="red")
plt.bar(x, women, bottom=men, label="Women", color="green")
plt.xlabel("Categories")
plt.ylabel("Scores")
plt.title("Stacked Bar Chart")
plt.xticks(x, categories)
plt.legend()
plt.show()
```

Output:

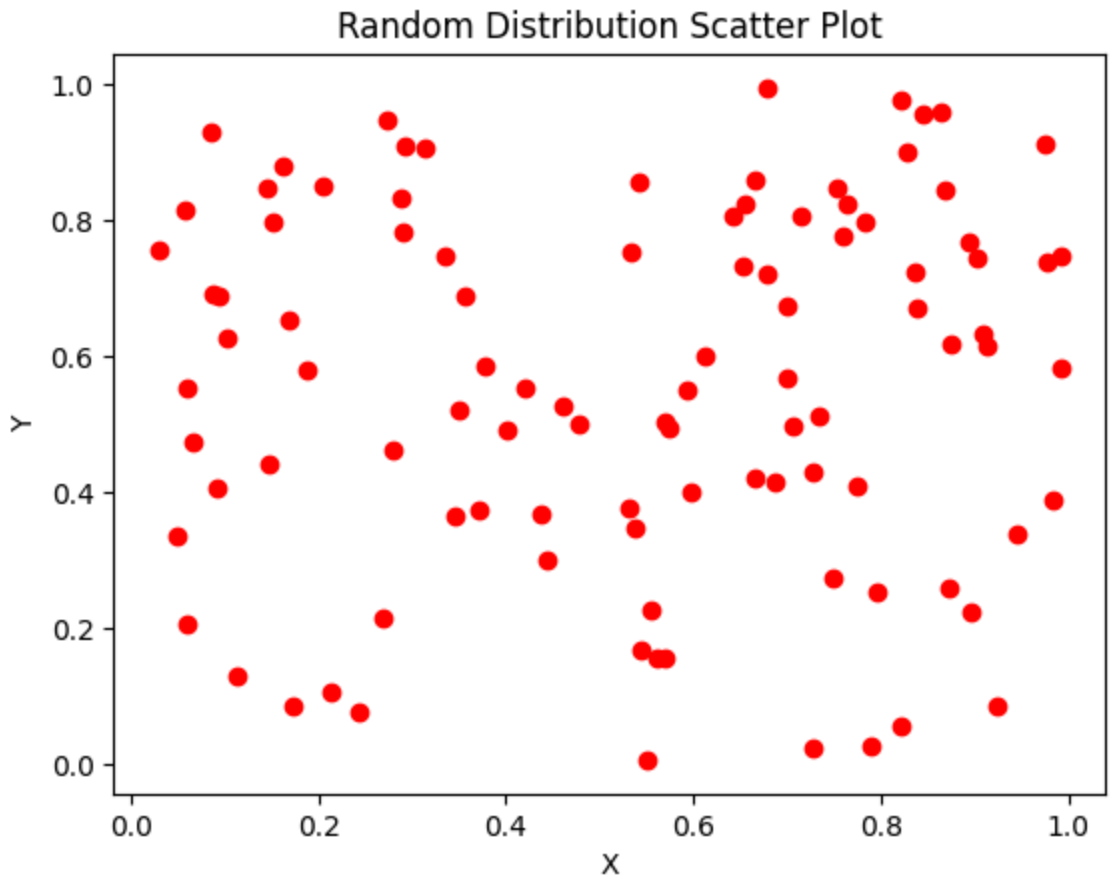


32. Scatter Plot Using Random Distribution

Python Code:

```
import numpy as np
import matplotlib.pyplot as plt
x = np.random.rand(100)
y = np.random.rand(100)
plt.scatter(x, y, color="red")
plt.xlabel("X")
plt.ylabel("Y")
plt.title("Random Distribution Scatter Plot")
plt.show()
```

Output:

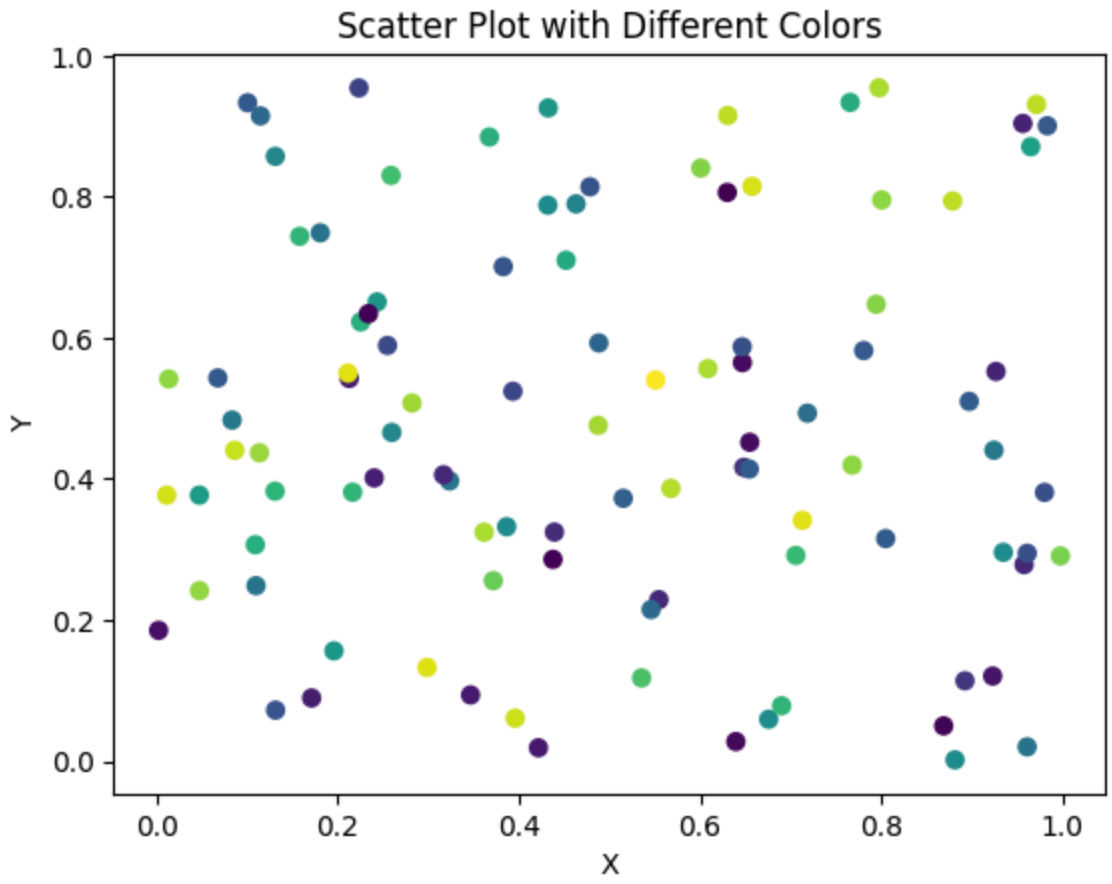


33. Scatter Plot with Different Colors

Python Code:

```
import numpy as np
import matplotlib.pyplot as plt
x = np.random.rand(100)
y = np.random.rand(100)
colors = np.random.rand(100)
plt.scatter(x, y, c=colors)
plt.xlabel("X")
plt.ylabel("Y")
plt.title("Scatter Plot with Different Colors")
plt.show()
```

Output:



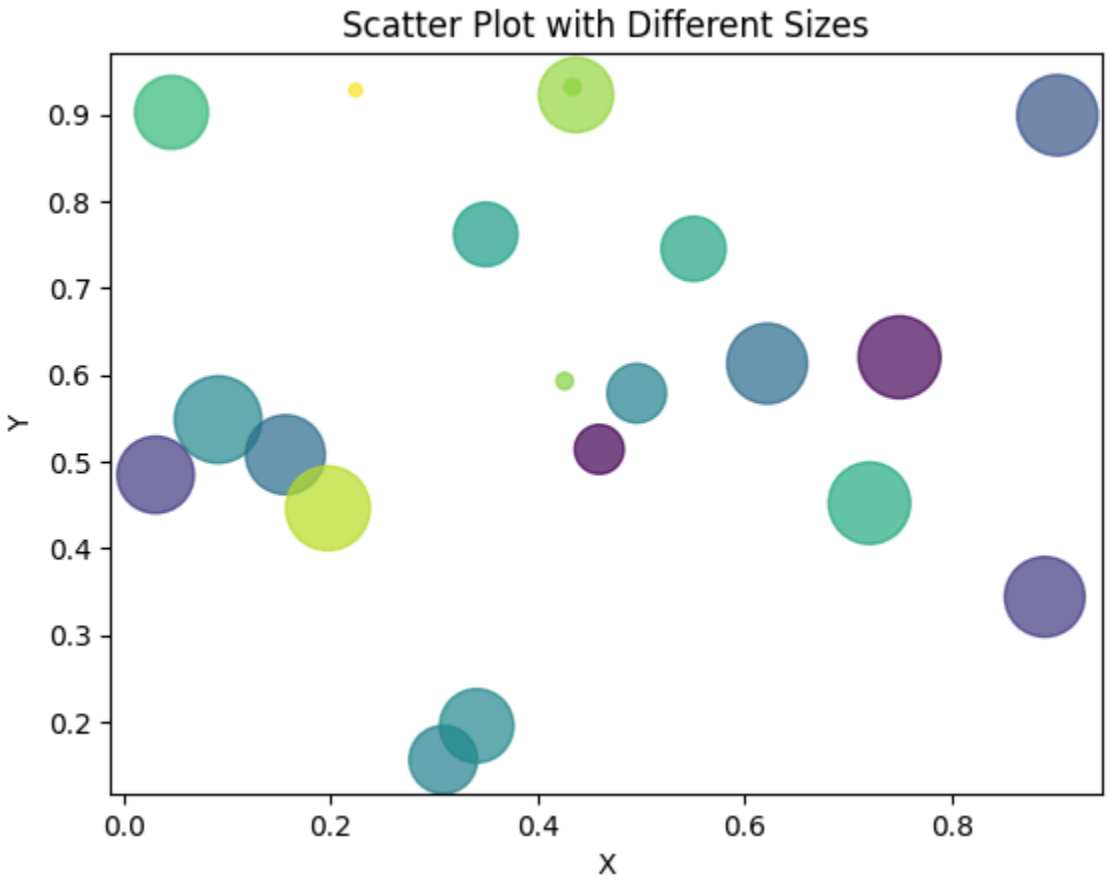
34. Scatter Plot with Different Sizes

Python Code:

```
import numpy as np
import matplotlib.pyplot as plt
x = np.random.rand(20)
y = np.random.rand(20)
colors = np.random.rand(20)
sizes = np.random.rand(20) * 1000
plt.scatter(
    x, y,
    c=colors,
    s=sizes,
    alpha=0.7
)
```

```
plt.xlabel("X")
plt.ylabel("Y")
plt.title("Scatter Plot with Different Sizes")
plt.show()
```

Output:



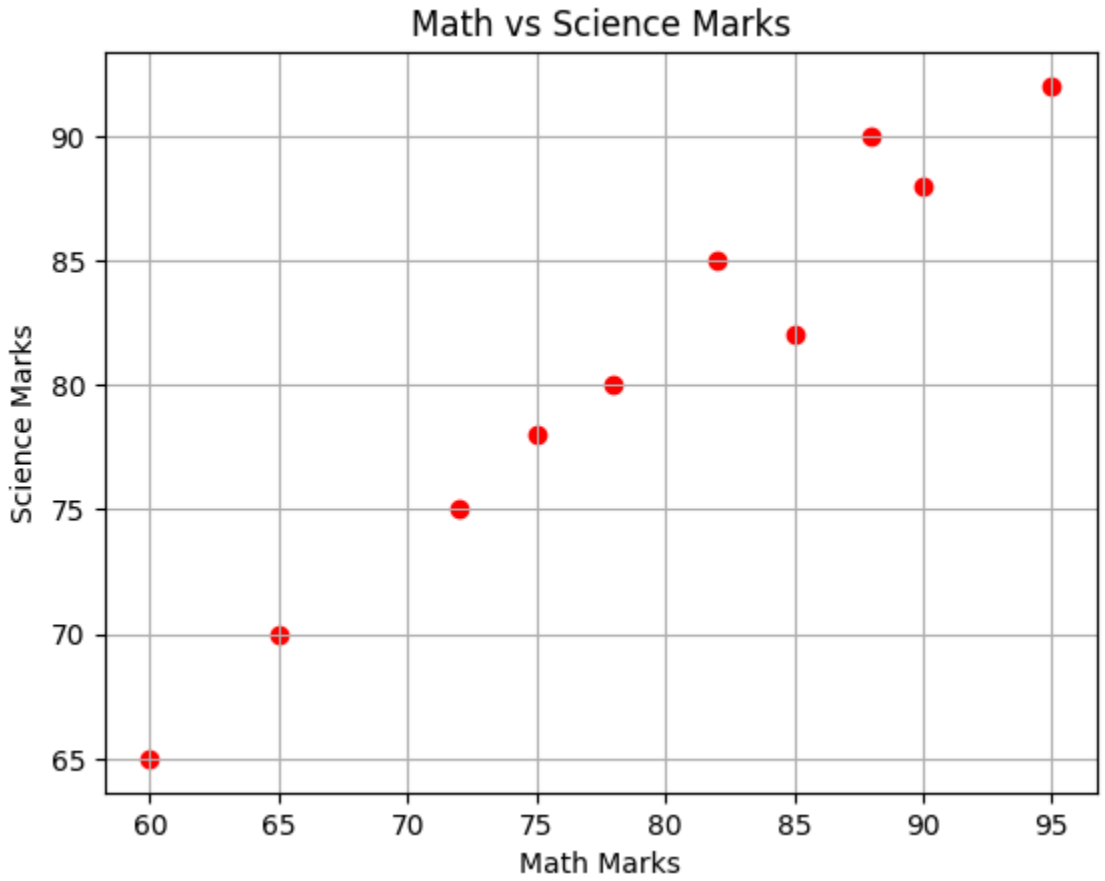
35. Scatter Plot Comparing Two Subjects

Python Code:

```
import matplotlib.pyplot as plt
math_marks = [78, 85, 90, 65, 72, 88, 95, 60, 82, 75]
science_marks = [80, 82, 88, 70, 75, 90, 92, 65, 85, 78]
plt.scatter(math_marks, science_marks, color="red")
plt.xlabel("Math Marks")
plt.ylabel("Science Marks")
```

```
plt.title("Math vs Science Marks")
plt.grid(True)
plt.show()
```

Output:



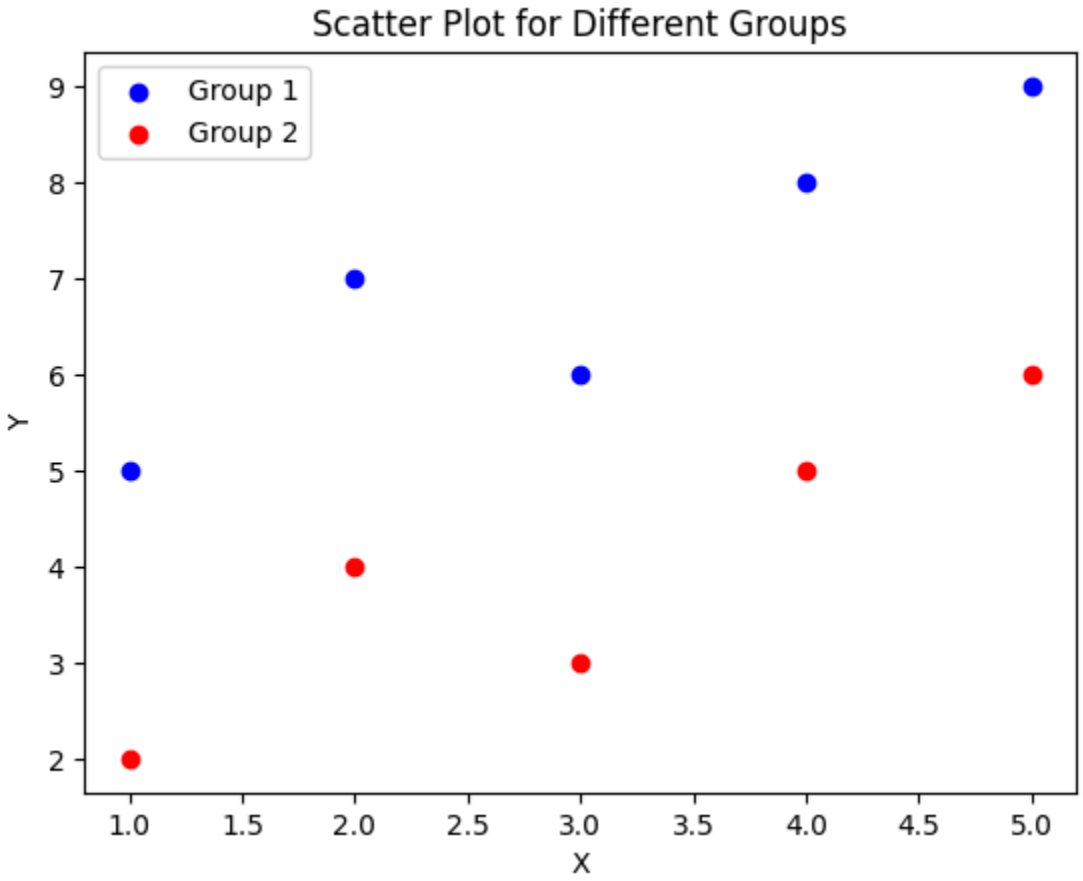
36. Scatter Plot for Different Groups

Python Code:

```
import matplotlib.pyplot as plt
group1_x = [1, 2, 3, 4, 5]
group1_y = [5, 7, 6, 8, 9]
group2_x = [1, 2, 3, 4, 5]
group2_y = [2, 4, 3, 5, 6]
plt.scatter(
    group1_x, group1_y,
    color="blue",
```

```
    label="Group 1"      )
plt.scatter(
    group2_x, group2_y,
    color="red",
    label="Group 2"     )
plt.xlabel("X")
plt.ylabel("Y")
plt.title("Scatter Plot for Different Groups")
plt.legend()
plt.show()
```

Output:



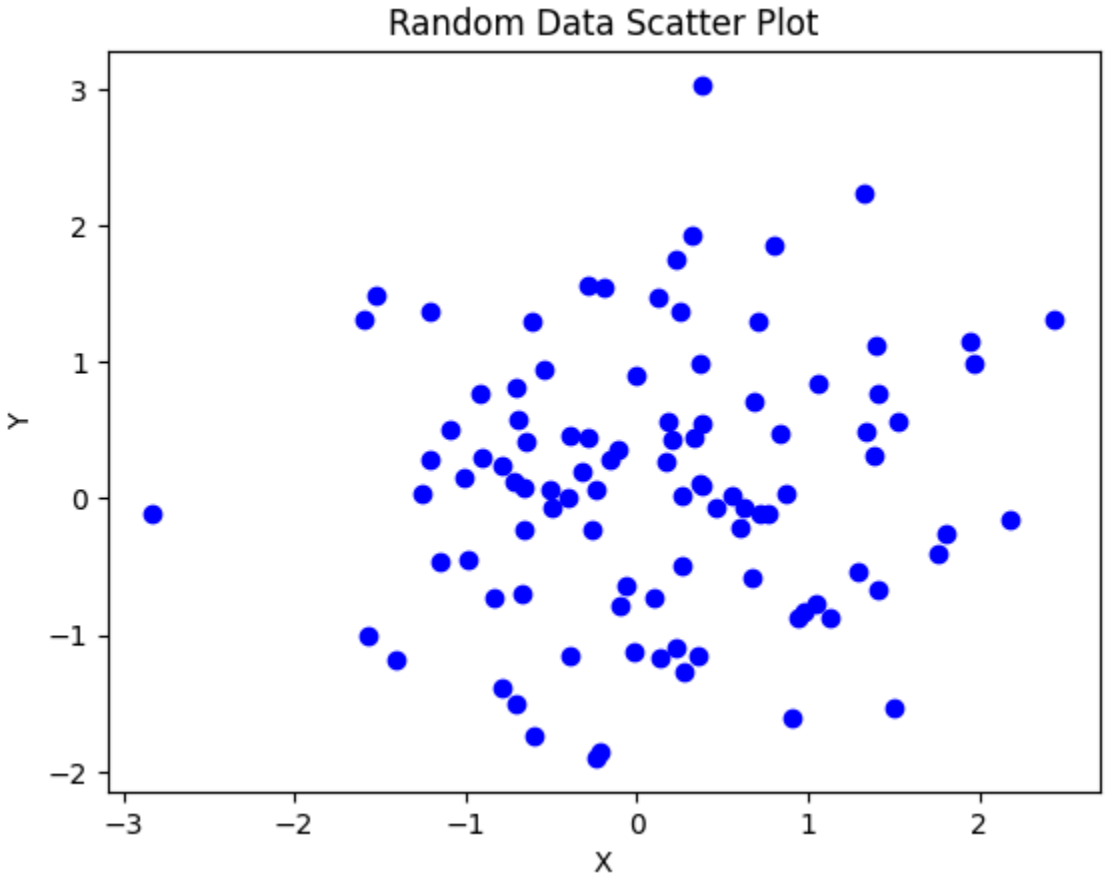
37. Scatter Plot with Random Data

Python Code:

```
import numpy as np
```

```
import matplotlib.pyplot as plt
x = np.random.randn(100)
y = np.random.randn(100)
plt.scatter(x, y, color="blue")
plt.xlabel("X")
plt.ylabel("Y")
plt.title("Random Data Scatter Plot")
plt.show()
```

Output:



38. Create a DataFrame from a Dictionary

Python Code:

```
import pandas as pd
data = {
```

```
    "Name": ["John", "Anna", "David", "Kiran"],
    "Age": [20, 21, 22, 20],
    "Gender": ["Male", "Female", "Male", "Female"]
}
df = pd.DataFrame(data)
print(df)
```

Output:

```
    Name  Age  Gender
0   John   20    Male
1   Anna   21  Female
2  David   22    Male
3  Kiran   20  Female
```

39. Get the First 3 Rows of a DataFrame

Python Code:

```
import pandas as pd
data = {
    "Name": ["John", "Anna", "David", "Kiran", "Rahul"],
    "Age": [20, 21, 22, 20, 23],
    "Course": ["Python", "Java", "C++", "Python", "Java"]
}
df = pd.DataFrame(data)
print(df.head(3))
```

Output:

```
    Name  Age  Course
0   John   20  Python
1   Anna   21    Java
2  David   22     C++
```

40. Locate Columns Using `loc`

Python Code:

```
import pandas as pd
```

```
data = {
    "Name": ["John", "Anna", "David", "Kiran"],
    "Age": [20, 21, 22, 20],
    "Course": ["Python", "Java", "C++", "Python"]
}
df = pd.DataFrame(data)
print(df.loc[:, ["Name", "Age"]])
```

Output: